Chapter IV

Exercise 00 : ft_iterative_factorial

	Exercise 00
ft_iterative_factorial	
Turn-in directory: $ex00/$	
Files to turn in: `ft_iterative_factorial.c`	
Allowed functions: `None`	

- Create an iterative function that returns a number. This number should be the result of a factorial operation based on the given parameter.
- If the argument is not valid, the function should return 0.
- Overflows do not need to be handled; the function's return value will be undefined in such cases.
- The function should be prototyped as follows:

```
int ft_iterative_factorial(int nb);
```

Chapter V

Exercise 01 : ft_recursive_factorial

	Exercise 01
ft_recursive_factorial	
Turn-in directory: $ex01/$	
Files to turn in: `ft_recursive_factorial.c`	
Allowed functions: `None`	

- Create a recursive function that returns the factorial of the given parameter.
- If the argument is not valid, the function should return 0.
- Overflows do not need to be handled; the function's return value will be undefined in such cases.
- The function should be prototyped as follows:

```
int ft_recursive_factorial(int nb);
```

Chapter VI

Exercise 02 : ft_iterative_power

	Exercise 02
ft_iterative_power	
Turn-in directory: $ex02/$	
Files to turn in: `ft_iterative_power.c`	
Allowed functions: `None`	

- Create an iterative function that returns the result of raising a number to a given power.
- If the power is less than 0, the function should return 0.
- Overflows do not need to be handled.
- By definition, 0 raised to the power of 0 should return 1.
- The function should be prototyped as follows:

```
int ft_iterative_power(int nb, int power);
```

Chapter VII

Exercise 03 : ft_recursive_power

	Exercise 03
ft_recursive_power	
Turn-in directory: $ex03/$	
Files to turn in: `ft_recursive_power.c`	
Allowed functions: `None`	

- Create a recursive function that returns the result of raising a number to a given power.
- If the power is less than 0, the function should return 0.
- Overflows do not need to be handled; the function's return value will be undefined in such cases.
- By definition, 0 raised to the power of 0 should return 1.
- The function should be prototyped as follows:

```
int ft_recursive_power(int nb, int power);
```

Chapter VIII

Exercise 04 : ft_fibonacci

	Exercise 04
ft_fibonacci	
Turn-in directory: *ex*04/	
Files to turn in: `ft_fibonacci.c`	
Allowed functions: `None`	

- Create a function `ft_fibonacci`, that returns the `n`-th element of the Fibonacci sequence, with the first element at index 0.
  The Fibonacci sequence will be considered to start as follows: 0, 1, 1, 2.
- Overflows do not need to be handled; the function's return value will be undefined in such cases.
- The function should be prototyped as follows:

```
int ft_fibonacci(int index);
```

- `ft_fibonacci` must be implemented recursively.
- If `index` is less than 0, the function should return -1.

Chapter IX

Exercise 05 : ft_sqrt

	Exercise 05
ft_sqrt	
Turn-in directory: $ex05/$	
Files to turn in: `ft_sqrt.c`	
Allowed functions: `None`	

- Create a function that returns the square root of a given number (if it exists), or 0 if the square root is an irrational number.
- The function should be prototyped as follows:

```
int ft_sqrt(int nb);
```

Chapter X

Exercise 06 : ft_is_prime

	Exercise 06
ft_is_prime	
Turn-in directory: $ex06/$	
Files to turn in: `ft_is_prime.c`	
Allowed functions: `None`	

- Create a function that returns 1 if the given number is a prime number and 0 if it is not.
- The function should be prototyped as follows:

```
int ft_is_prime(int nb);
```

`0 and 1 are not prime numbers.`

Chapter XI

Exercise 07 : ft_find_next_prime

	Exercise 07
ft_find_next_prime	
Turn-in directory: $ex07/$	
Files to turn in: `ft_find_next_prime.c`	
Allowed functions: `None`	

- Create a function that returns the next prime number greater than or equal to the given number.
- The function should be prototyped as follows:

```
int ft_find_next_prime(int nb);
```

Chapter XII

Exercise 08 : The Ten Queens

	Exercise 08
The Ten Queens	
Turn-in directory: $ex08/$	
Files to turn in: `ft_ten_queens_puzzle.c`	
Allowed functions: `write`	

- Create a function that displays all possible placements of ten queens on a 10×10 chessboard, ensuring that no two queens can attack each other in a single move. The function should return the total number of valid solutions.
- Recursion is required to solve this problem.
- The function should be prototyped as follows:

```
int ft_ten_queens_puzzle(void);
```

- Output format:

```
$>./a.out | cat -e
0257948136$
0258693147$
...
4605713829$
4609582731$
...
9742051863$
$>
```

- The sequence is read from left to right, where:
  - The first digit represents the row position of the queen in the first column (index starting at 0).
  - The Nth digit represents the row position of the queen in the Nth column.
- The function should return the total number of valid solutions found.